

Habib University
Dhanani School of Science and Engineering
Computer Architecture

EE 371 / CS 330 / CE 321 (Spring 2026)

Homework 3

Release date: 27th March 2026 **Due date:**
Total marks: 100 **Obtained Marks:**
Student Name: **Student ID:**

Instructions:

1. This assignment should be done in pairs.
2. All questions should be answered in **black ink only**.
3. Scan your answer sheet and upload it on HU LMS before the due date.
4. Take a print of the assignment and solve (provide answers) in the space provided after every question. You can use extra sheets for your answers. Attach them properly.
5. You can also be asked to give a viva where you will be judged whether you understood the question yourself or not. If you are unable to correctly answer the question you have attempted right, you may lose your marks.
6. Zero will be given if the assignment is found to be plagiarized.
7. No assignment shall be graded if submitted late.

CLO Assessment:

This assignment assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	<i>Explain</i> the role of ISA in modern processors and instruction encodings and assembly language programming	
CLO 2	<i>Explain</i> the architecture and working of a single cycle processor	
CLO 3	<i>Design</i> the architecture to mitigate issues of a pipelined processor	✓
CLO 4	<i>Analyze</i> the performance of cache operations	

Question # 1 (Points: 12)

A 5-stage pipelined processor has the following stage latencies:

IF	ID	EX	MEM	WB
220ps	200ps	320ps	260ps	180ps

and also consider the inter-stage register delay to be 25ps.

Also, assume that instructions executed by the processor for a program-A are divided as follows:

ALU/Logic	Jump/Branch	Load	Store
40%	10%	25%	25%

1. Compute the clock cycle time for both pipelined and non-pipelined versions. (2 points)
2. Determine the total latency for a store instruction in both processors. (2 points)
3. If 1200 instructions are executed with no hazards, calculate the speedup achieved. (2 points)

- Now assume a stall occurs every five instructions. What is the new speedup? (2 points)
- Suggest a possible architectural modification to reduce overall cycle time without changing ISA behavior. Calculate the new pipeline clock cycle time. (2 points)
- Assuming all stages are fully utilized, calculate the utilization of the instruction memory and data memory over 1000 instruction executions. (2 points)

Question # 2 (Points: 10)

In the RISC-V single-cycle processor design, load (**ld**) is the instruction with the longest latency. If we modify the memory instructions so that there is no address calculation (i.e., the address to be loaded from/stored to must be calculated and placed in **rs1** before calling **ld/sd**), then no instruction would use both the ALU and Data memory. This would allow us to reduce the clock cycle time of the single-cycle implementation. However, it would also increase the number of instructions, because many **ld** and **sd** instructions would need to be replaced with **ld/add** or **sd/add** combinations.

$$\begin{array}{lcl} \text{ld} & \text{x18, 100(x5)} & \Rightarrow \text{addi} \quad \text{x5, x0, 100} \\ & & \text{ld} \quad \text{x18, x5} \end{array}$$

1. For the latencies as mentioned in Question 1, what would the new clock cycle time be for single-cycle (non-pipelined) processor? (1 mark)
2. In the non-pipelined version, would a program with the instruction mix presented in Question 1 run faster or slower on this new CPU? And by how much? (For simplicity, assume every **ld** and **sd** instruction is replaced with a sequence of two instructions.) (1 mark)
3. What is the primary factor that influences whether a program will run faster or slower on the new CPU? (1 mark)

Question # 3 (Points: 10)

Consider the fragment of RISC-V assembly below:

```
label:  ld    x31, 32(x10)
        sd    x11, 8(x10)
        sub   x12, x14, x13
        add   x15, x12, x13
        beq   x24, x0, label
        add   x31, x31, x14
```

Suppose we modify the pipeline so that it has only one memory (that handles both instructions and data). In this case, there will be a structural hazard every time a program needs to fetch an instruction during the same cycle in which another instruction accesses data.

1. Draw a pipeline diagram to show where the above code will stall due to structural hazards. (3 marks)
2. In general, is it possible to reduce the number of stalls/NOPs resulting from this structural hazard by reordering code? (2 marks)
3. We have seen that some data hazards can be eliminated by adding NOPs to the code. Can we do the same with this structural hazard? If so, explain how. If not, explain why not. (2 marks)

4. Approximately how many stalls would you expect this structural hazard to generate in a typical program? Use the instruction mix from Question 1. (3 marks)

Question # 4 (Points: 15)

Suppose one of the following control signals in a single-cycle RISC-V processor is stuck-at-1 (always logic 1). Which instruction(s) would malfunction, what would be the issues observed and why? Refer to the extended single-cycle datapath discussed in class.

1. RegWrite (1.8 marks)

2. MemRead (1.8 marks)

3. MemWrite (1.8 marks)

4. Branch (1.8 marks)

5. ALUSrc (1.8 marks)

6. PCSrc (1.8 marks)

7. ALUOp₁ (1.8 marks)

8. ALUOp₀ (1.8 marks)

Question # 5 (Points: 13)

The pipelined RISC-V processor executes the following code segment:

```
addi x1, x0, 5      # x1 = 5
addi x2, x0, 10     # x2 = 10
add  x3, x1, x2      # x3 = 15
sub  x4, x3, x1      # x4 = 10
lw   x5, 0(x4)       # load word from memory address in x4
add  x6, x5, x1      # x6 = x5 + x1
sw   x6, 4(x4)       # store word from x6 to memory[x4 + 4]
add  x7, x6, x3      # x7 = x6 + x3
```

Assume the following initial conditions:

- All registers are initialized to 0 except x0 (which is always 0).
- Memory location Mem[10] = 40.

1. Identify all data hazards present in the above sequence (specify the dependent registers and the type of hazard). (3 marks)
2. Assuming there is no forwarding and no stall detection, complete a table showing the value of each register (x1–x7) after each instruction finishes execution. Clearly indicate where incorrect values would appear due to hazards. (4 marks)

3. Describe how forwarding and hazard detection unit could be used to ensure correct execution of this program. Mention which hazards are resolved by forwarding and which require stalls. (3 marks)

4. Redraw the pipeline diagram assuming full forwarding and hazard detection. Clearly label where stalls (if any) occur. (3 marks)

Question # 6 (Points: 15)

Consider the following instruction that is not found in the RISC-V architecture:

Add to Memory

```
addm rd, Offset(rs)      // Reg[rd] = Reg[rd] +  
                          //      Mem[Offset + Reg[rs]]
```

We want to modify the RISC-V processor to support the above instruction. For parts (b) and (d) below, you can use a printed version of the figures in the book over which you can draw your suggested modifications (no need to draw the entire diagram from scratch).

1. Suggest if any of the existing instruction formats is a good choice to encode the new instruction. If not, then propose a new instruction format. (2 marks)
2. Modify the datapath and control signals of the single-cycle RISC-V processor (Figure 4.17 of the book) to execute the new instruction using the instruction format suggested in part A. Use the minimal amount of additional hardware and clock cycles/control states. Remember when adding new instructions, don't break the operation of the standard ones. (4 marks)
3. Discuss the effect of the modification in part (B) on the latency of single-cycle non-pipelined CPU. (2 marks)

- [illegible]

Question # 7 (Points: 10)

The pipelined processor's performance might be better if branches take place during the Decode stage rather than the Execute stage. Show how to modify the pipelined processor to move the branch logic to the Decode stage. How do the stall, flush, and forwarding signals change? You can use a printed version of the figures in the book over which you can draw your suggested modifications.

Question # 8 (Points: 15)

In a pipelined CPU, the multiply instruction (`mul`) requires two cycles in the EX stage (EXa and EXb).

1. Explain why this situation causes a structural hazard in the pipeline. Use a pipeline timing chart for a sequence containing two consecutive `mul` instructions to illustrate the conflict. (Assume there is no stall unit.) (6 marks)
2. Clearly highlight or annotate the modifications needed to support a two-cycle `mul` instruction using Figure 4.58 from the textbook. Indicate any new control signals, hardware components, or changes to existing control logic that prevent this hazard. (6 marks)
3. Briefly describe how your proposed modifications resolve the hazard. (3 marks)